## 1.Create a bash script to check if a directory is available or not.

```
[ec2-user@ip-172-31-22-105 shells]$ nano check-directory.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod +x check-directory.bash
[ec2-user@ip-172-31-22-105 shells]$ ./check-directory.bash
Enter the directory path:
/home/ec2-user/shells
Directory exists!
[ec2-user@ip-172-31-22-105 shells]$ ./check-directory.bash
Enter the directory path:
/home/ec2-user/unknownfolder
Directory does not exist!
[ec2-user@ip-172-31-22-105 shells]$ cat check-directory.bash
#!/bin/bash

echo "Enter the directory path:"
read dir

if [ -d "$dir" ]
then
    echo "Directory exists!"
else
    echo "Directory does not exist!"
fi
[ec2-user@ip-172-31-22-105 shells]$
```

1. I navigated to the `shells` directory.
2. I created a Bash script named `check-directory.bash`.
3. I wrote the Bash code to check whether a directory exists.
4. I saved the script and exited the Nano editor.
5. I gave execute permission to the script.
6. I executed the Bash script.
7. I entered the path of an existing directory.
8. I verified that the message “Directory exists!” was displayed.
9. I entered the path of a non-existing directory.
10. I verified that the message “Directory does not exist!” was displayed.
11. I captured screenshots of the script and its outputs for documentation.

## 2.Create a bash script to create multiple files.

```
[ec2-user@ip-172-31-22-105 shells]$ vi create-files.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 create-files.bash
[ec2-user@ip-172-31-22-105 shells]$ ./create-files.bash
Enter the number of files to create:
3
file1.txt created successfully
file2.txt created successfully
file3.txt created successfully
All files created!
[ec2-user@ip-172-31-22-105 shells]$ cat create-files.bash
#!/bin/bash

echo "Enter the number of files to create:"
read n

for ((i=1; i<=n; i++))
do
    touch "file$i.txt"
    echo "file$i.txt created successfully"
done

echo "All files created!"
[ec2-user@ip-172-31-22-105 shells]$ ls -l file*.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Oct 16 10:19 file1.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Oct 16 10:19 file2.txt
-rw-r--r--. 1 ec2-user ec2-user 0 Oct 16 10:19 file3.txt
[ec2-user@ip-172-31-22-105 shells]$
```

1. I navigated to the `shells` directory.
2. I created a Bash script named `create-files.bash`.
3. I wrote the Bash code to create multiple files using a loop.
4. I saved the script and exited the vi editor.
5. I gave execute permission to the script.
6. I executed the Bash script.
7. I entered the number of files to create.
8. I verified that the files were created successfully.
9. I listed the files using the `ls` command.
10. I captured screenshots of the script and its output for documentation.

## 3.Create a bash script to take a backup of a directory.

```
[ec2-user@ip-172-31-22-105 shells]$ vi backup-directory.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 ^C
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 backup-directory.bash
[ec2-user@ip-172-31-22-105 shells]$ ./backup-directory.bash
Enter the directory path to back up:
^C
[ec2-user@ip-172-31-22-105 shells]$ pwd
/home/ec2-user/shells
[ec2-user@ip-172-31-22-105 shells]$ ./backup-directory.bash
Enter the directory path to back up:
/home/ec2-user/shells
tar: Removing leading `/' from member names
Backup created successfully: backup_20261016_103158.tar.gz
[ec2-user@ip-172-31-22-105 shells]$ ls -lh backup_20261016_103158.tar.gz
-rw-r--r--. 1 ec2-user ec2-user 1.6K Oct 16 10:31 backup_20261016_103158.tar.gz
[ec2-user@ip-172-31-22-105 shells]$ cat backup-directory.bash
#!/bin/bash

echo "Enter the directory path to back up:"
read source_dir

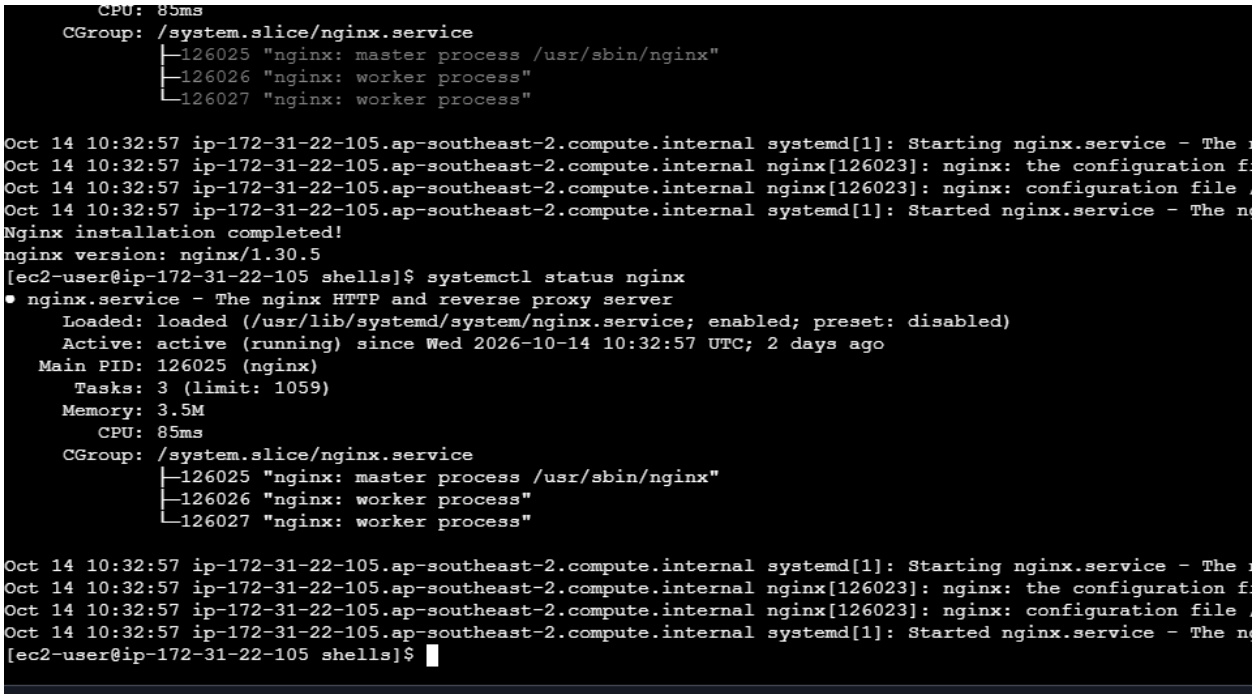
if [ -d "$source_dir" ]
then
    backup_file="backup_$(date +%Y%m%d_%H%M%S).tar.gz"
    tar -czf "$backup_file" "$source_dir"
    echo "Backup created successfully: $backup_file"
else
    echo "Directory does not exist!"
fi
[ec2-user@ip-172-31-22-105 shells]$
```

1. I navigated to the `shells` directory.
2. I created a Bash script named `backup-directory.bash`.
3. I wrote the Bash code to back up a directory using the `tar` command.
4. I saved the script and exited the vi editor.
5. I gave execute permission to the script.
6. I executed the Bash script.
7. I entered the directory path `/home/ec2-user/shells`.
8. I verified that the backup archive was created successfully.
9. I checked the backup file and its contents.
10. I captured screenshots of the script and its output for documentation.

## 4.Create a bash script to install Nginx on an EC2 server.

```
[ec2-user@ip-172-31-22-105 shells]$ vi  install-nginx.bash
[ec2-user@ip-172-31-22-105 shells]$ ./install-nginx.bash
Installing Nginx...
Last metadata expiration check: 0:09:53 ago on Fri Oct 16 10:39:23 2026.
Package nginx-1:1.30.5-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
Checking Nginx status...
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
     Active: active (running) since Wed 2026-10-14 10:32:57 UTC; 2 days ago
   Main PID: 126025 (nginx)
      Tasks: 3 (limit: 1059)
     Memory: 3.5M
        CPU: 85ms
     CGroup: /system.slice/nginx.service
             ├─126025 "nginx: master process /usr/sbin/nginx"
             ├─126026 "nginx: worker process"
             └─126027 "nginx: worker process"

Oct 14 10:32:57 ip-172-31-22-105.ap-southeast-2.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Oct 14 10:32:57 ip-172-31-22-105.ap-southeast-2.compute.internal nginx[126023]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Oct 14 10:32:57 ip-172-31-22-105.ap-southeast-2.compute.internal nginx[126023]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Oct 14 10:32:57 ip-172-31-22-105.ap-southeast-2.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
Nginx installation completed!
nginx version: nginx/1.30.5
```




1.I navigated to the `shells` directory.

2.I created a Bash script named `install-nginx.bash`.

3.I wrote the Bash code to install Nginx using the `dnf` package manager.

4.I saved the script and exited the Nano editor.

5.I gave execute permission to the script.

6.I executed the Bash script.

7.I installed Nginx on the EC2 server.

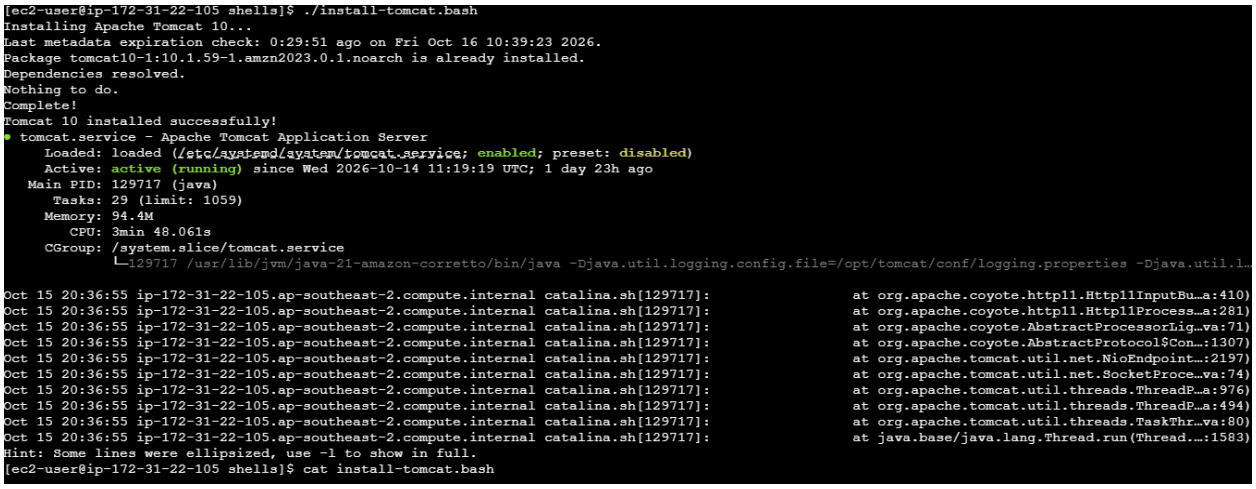
8.I tested the Nginx configuration for errors.

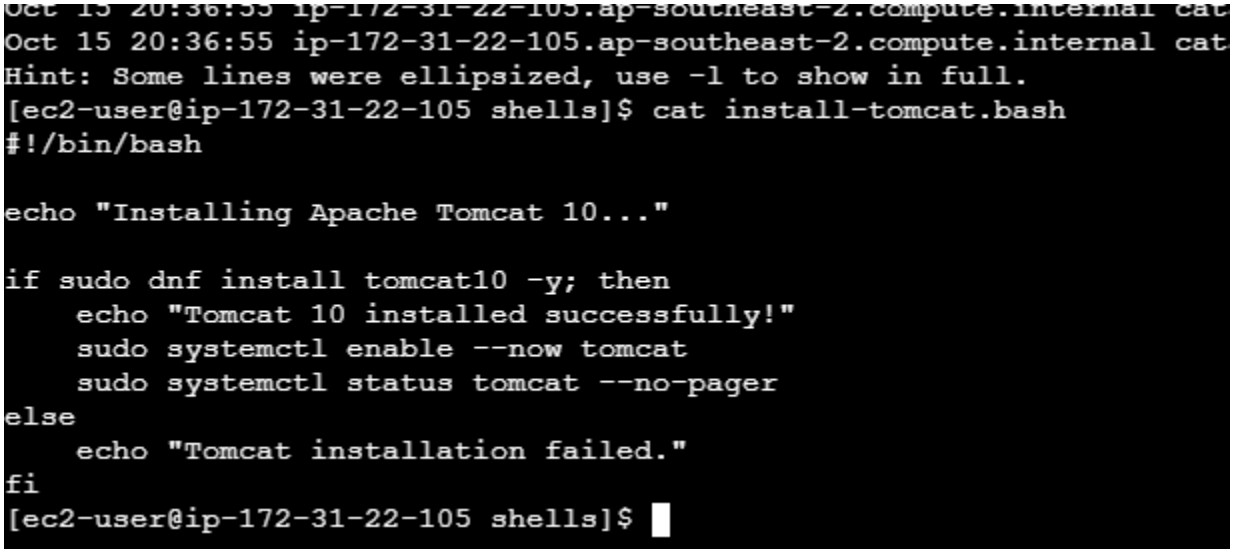
9.I enabled and started the Nginx service.

10.I verified that Nginx was running successfully.

11.I captured screenshots of the script and its output for documentation.

## 5.Create a bash script to install Apache Tomcat on an EC2 server



1. I navigated to the `shells` directory.
2. I created a Bash script named `install-tomcat.bash`.
3. I wrote the Bash code to install Apache Tomcat using the `dnf` package manager.
4. I saved the script and exited the vi editor.
5. I gave execute permission to the script.
6. I executed the Bash script.
7. I encountered a package conflict because Tomcat 10 was already installed and conflicted with Tomcat 9.
8. I checked the installed Tomcat packages.
9. I modified the script to install Tomcat 10 using `dnf`.
10. I executed the updated script.
11. I verified the Tomcat service status.
12. I captured screenshots of the script and its output for documentation.

## 6.Create a bash script to check if the Nginx service is running, if not running then script should start the service

```
[ec2-user@ip-172-31-22-105 shells]$ vi check-nginx.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 check-nginx.bash
[ec2-user@ip-172-31-22-105 shells]$ ./check-nginx.bash
Checking Nginx service status...
Nginx is already running.
[ec2-user@ip-172-31-22-105 shells]$ sudo systemctl stop nginx
[ec2-user@ip-172-31-22-105 shells]$ ./check-nginx.bash
Checking Nginx service status...
Nginx is not running. Starting Nginx...
Nginx started successfully!
[ec2-user@ip-172-31-22-105 shells]$ cat check-nginx.bash
#!/bin/bash

echo "Checking Nginx service status..."

if systemctl is-active --quiet nginx
then
    echo "Nginx is already running."
else
    echo "Nginx is not running. Starting Nginx..."
    sudo systemctl start nginx

    if systemctl is-active --quiet nginx
    then
        echo "Nginx started successfully!"
    else
        echo "Failed to start Nginx."
    fi
fi
[ec2-user@ip-172-31-22-105 shells]$
```

1.I navigated to the `shells` directory.

2.I created a Bash script named `check-nginx.bash`.

3.I wrote the Bash code to check the Nginx service status.

4.I configured the script to start Nginx if it was not running.

5.I saved the script and exited the Nano editor.

6.I gave execute permission to the script.

7.I executed the Bash script.

8.I verified that the script detected the Nginx service status.

9.I tested whether the script could start Nginx when it was stopped.

10.I captured screenshots of the script and its output for documentation.

## 7.Create a bash script for a calculator.

```
[ec2-user@ip-172-31-22-105 shells]$ vi calculator.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 calculator.bash
[ec2-user@ip-172-31-22-105 shells]$ ./calculator.bash
Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice: 2
Enter first number: 10
Enter second number: 5
Result: 5
[ec2-user@ip-172-31-22-105 shells]$ cat calculator.bash
#!/bin/bash

echo "Simple Calculator"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"

read -p "Enter your choice: " choice
read -p "Enter first number: " num1
read -p "Enter second number: " num2

case $choice in
    1) result=$((num1 + num2))
       echo "Result: $result" ;;
    2) result=$((num1 - num2))
       echo "Result: $result" ;;
```

```
[ec2-user@ip-172-31-22-105 shells]$ cat calculator.bash
#!/bin/bash

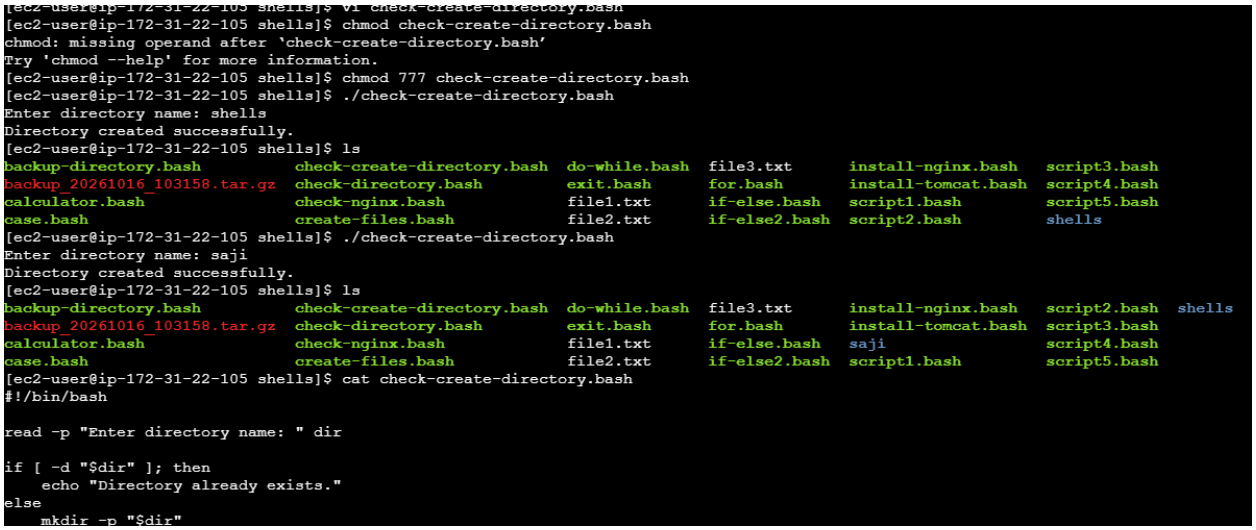
echo "Simple Calculator"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"

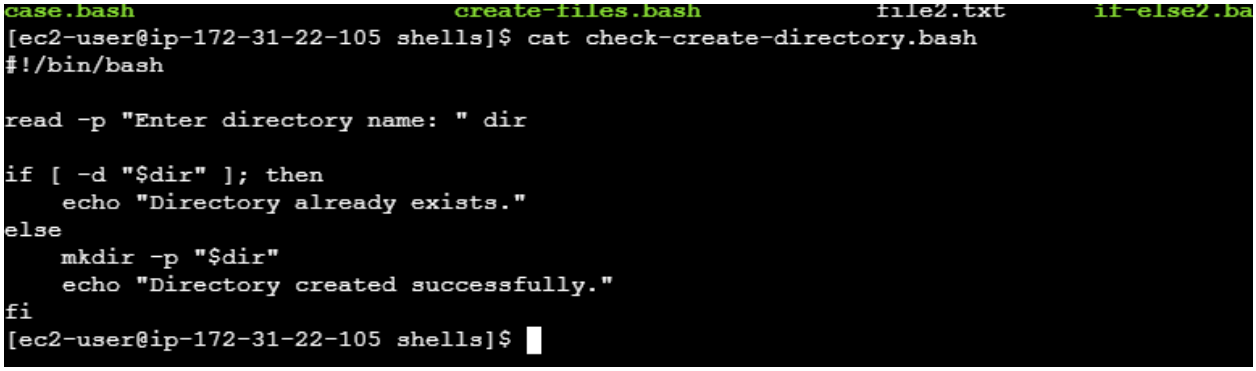
read -p "Enter your choice: " choice
read -p "Enter first number: " num1
read -p "Enter second number: " num2

case $choice in
    1) result=$((num1 + num2))
       echo "Result: $result" ;;
    2) result=$((num1 - num2))
       echo "Result: $result" ;;
    3) result=$((num1 * num2))
       echo "Result: $result" ;;
    4) if [ "$num2" -eq 0 ]; then
           echo "Error: Division by zero is not allowed."
       else
           result=$((num1 / num2))
           echo "Result: $result"
       fi ;;
    *) echo "Invalid choice!" ;;
esac
[ec2-user@ip-172-31-22-105 shells]$
```

1. I navigated to the `shells` directory.
2. I created a Bash script named `calculator.bash`.
3. I wrote the Bash code to perform addition, subtraction, multiplication, and division.
4. I saved the script and exited the Nano editor.
5. I gave execute permission to the script.
6. I executed the Bash calculator script.
7. I selected an arithmetic operation.
8. I entered two numbers and verified the result.
9. I checked the division-by-zero condition.
10. I captured screenshots of the script and its output for documentation.

# 8.Create a bash script to check if a directory exists, if not then create a directory




1. I navigated to the existing `shells` directory.
2. I created a Bash script named `check-create-directory.bash`.
3. I wrote the code to check whether a directory exists.
4. I added a condition to create the directory if it does not exist.
5. I saved the script and exited the Nano editor.
6. I gave execute permission to the script using `chmod +x`.
7. I executed the Bash script.
8. I entered a directory name and verified that the directory was created.
9. I executed the script again and confirmed that it detected the existing directory.
10. I captured screenshots of the script and its output for documentation.

## 9.Create a bash script to delete the last 3 lines of a file.

```
[ec2-user@ip-172-31-22-105 shells]$ vi sample.txt
[ec2-user@ip-172-31-22-105 shells]$ ./delete-last-3-lines.bash
Enter file name: sample.txt
Last 3 lines deleted successfully.
[ec2-user@ip-172-31-22-105 shells]$ cat sample.txt
line 1
line 2
line 3
[ec2-user@ip-172-31-22-105 shells]$ vi test
[ec2-user@ip-172-31-22-105 shells]$ cat test
1
2
3
4
5
6
7

[ec2-user@ip-172-31-22-105 shells]$ ./delete-last-3-lines.bash
Enter file name: test
Last 3 lines deleted successfully.
[ec2-user@ip-172-31-22-105 shells]$ cat test
1
2
3
4
5
[ec2-user@ip-172-31-22-105 shells]$
```

```
4
5
[ec2-user@ip-172-31-22-105 shells]$ cat delete-last-3-lines.bash
#!/bin/bash

read -p "Enter file name: " file

if [ -f "$file" ]; then
    if [ "$(wc -l < "$file")" -lt 3 ]; then
        echo "Error: File has fewer than 3 lines."
    else
        head -n -3 "$file" > "${file}.tmp" &&
        mv "${file}.tmp" "$file"
        echo "Last 3 lines deleted successfully."
    fi
else
    echo "Error: File does not exist."
fi
[ec2-user@ip-172-31-22-105 shells]$
```

1.I navigated to the existing `shells` directory.

2.I created a Bash script named `delete-last-3-lines.bash`.

3.I wrote the code to check whether the specified file exists.

4.I added logic to remove the last three lines from the file.

5.I saved the script and exited the Nano editor.

6.I gave execute permission to the script.

7.I created a sample text file containing five lines.

8.I executed the Bash script and entered the sample file name.

9.I verified that only the first two lines remained.

10.I captured screenshots of the script and its output for documentation.

## 10.Bash script to monitor cpu and if it is more than 80% then send email notification.

```bash
#!/bin/bash

CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')

THRESHOLD=80
EMAIL="your-email@example.com"

echo "Current CPU usage: ${CPU}%"

if (( ${CPU%.*} > THRESHOLD )); then
    echo "CPU usage is ${CPU}% on $(hostname)" | mail -s "High CPU Usage Alert" "$EMAIL"
    echo "CPU usage is ${CPU}%. Email notification attempted."
else
    echo "CPU usage is ${CPU}%. CPU usage is normal."
fi
```

```
[ec2-user@ip-172-31-22-105 shells]$ vi monitor-cpu.bash
[ec2-user@ip-172-31-22-105 shells]$ ./monitor-cpu.bash
Current CPU usage: 3.1%
CPU usage is 3.1%. CPU usage is normal.
```

I navigated to the existing `shells` directory.

I created a Bash script named `monitor-cpu.bash`.

I wrote the code to retrieve CPU usage using the `top` command.

I set the CPU usage threshold to 80%.

I configured the recipient email address for notifications.

I added an `if-else` condition to check whether CPU usage exceeds 80%.

I added email notification logic to alert when CPU usage exceeds the threshold.

I saved the script and gave it execute permission using `chmod +x`.

I executed the script and verified the CPU usage output.

I confirmed that the script displayed the appropriate message based on CPU usage and captured a screenshot for documentation

## 11.Bash script to monitor disk space and if it is more than 80% then send email notification.

```
[ec2-user@ip-172-31-22-105 shells]$ vi  monitor-disk.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 monitor-disk.bash
[ec2-user@ip-172-31-22-105 shells]$ ./monitor-disk.bash
Disk usage on /sys/firmware/efi/efivars is 3%
Disk usage on / is 35%
Disk usage on /boot/efi is 13%
[ec2-user@ip-172-31-22-105 shells]$
```

```
#!/bin/bash

THRESHOLD=80
EMAIL="your-email@example.com"

df -P -x tmpfs -x devtmpfs | awk 'NR > 1 {print $5, $6}' |
while read -r USAGE MOUNT
do
    DISK_USAGE=${USAGE%\%}

    echo "Disk usage on $MOUNT is ${DISK_USAGE}%"

    if [ "$DISK_USAGE" -gt "$THRESHOLD" ]; then
        echo "Warning: Disk usage on $MOUNT is ${DISK_USAGE}%." |
            mail -s "High Disk Usage Alert" "$EMAIL"

        echo "Disk usage exceeds ${THRESHOLD}%. Email notification attempted."
    fi
done
~
```

1. I navigated to the existing `shells` directory.
2. I created a Bash script named `monitor-disk.bash`.
3. I wrote the code to check disk usage using the `df` command.
4. I set the disk usage threshold to 80%.
5. I configured the recipient email address for notifications.
6. I added a condition to identify filesystems exceeding the threshold.
7. I added email notification logic for high disk usage.
8. I saved the script and gave it execute permission using `chmod +x`.
9. I executed the script and verified disk usage using `df -h`.

10. I captured screenshots of the script and terminal output for documentation.

## 12.Bash script to monitor memory and if it is more than 80% then send email notification.

```
[ec2-user@ip-172-31-22-105 shells]$ vi monitor-disk.bash
[ec2-user@ip-172-31-22-105 shells]$ vi monitor-memory.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 monitor-memory.bash
[ec2-user@ip-172-31-22-105 shells]$ ./monitor-memory.bash
Current memory usage: 82%
./monitor-memory.bash: line 15: mail: command not found
Memory usage exceeds 80%. Email notification attempted.
[ec2-user@ip-172-31-22-105 shells]$
```

```
#!/bin/bash

THRESHOLD=80
EMAIL="your-email@example.com"

MEMORY_USAGE=$(free | awk '/Mem:/ {
    printf "%.0f", ($3 / $2) * 100
}')

echo "Current memory usage: ${MEMORY_USAGE}%"

if [ "$MEMORY_USAGE" -gt "$THRESHOLD" ]; then
    echo "Warning: Memory usage is ${MEMORY_USAGE}% on $(hostname)." |
        mail -s "High Memory Usage Alert" "$EMAIL"

    echo "Memory usage exceeds ${THRESHOLD}%. Email notification attempted."
else
    echo "Memory usage is normal."
fi
```

I navigated to the existing `shells` directory.

I created a Bash script named `monitor-memory.bash`.

I wrote the code to retrieve memory statistics using the `free` command.

I calculated memory usage as a percentage of total memory.

I set the memory usage threshold to 80%.

I configured the recipient email address for notifications.

I added a condition to send an email notification when memory usage exceeds 80%.

I saved the script and gave it execute permission using `chmod +x`.

I executed the script and verified memory usage using `free -h`.

I captured screenshots of the script and terminal output for documentation.

13.

# 14.Execute all scripts from the devops-cloud-documents- channel.

## 14.1 IF-ELSE CONDITION

1. Start the Bash script.
2. Assign the value 100 to the variable count.
3. Check the condition to see whether the value of count is equal to 100.
4. If the condition is TRUE, display "Count is 100".
5. If the condition is FALSE, execute the else part and display "Count is not 100".
6. End the IF-ELSE statement and finish the script.

```
[ec2-user@ip-172-31-22-105 shells]$ vi if-else.bash
[ec2-user@ip-172-31-22-105 shells]$ cat if-else.bash
#!/bin/bash

count=100
if [$count -eq 100]
then
        echo count is 100
else
        echo count is not 100
fi
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 if-else.bash
[ec2-user@ip-172-31-22-105 shells]$ ./if-else.bash
./if-else.bash: line 4: [100: command not found
count is not 100
[ec2-user@ip-172-31-22-105 shells]$ vi if-else.bash
[ec2-user@ip-172-31-22-105 shells]$ ./if-else.bash
count is 100
```

## 14.2 IF-ELSE FILE EXIST

```
[ec2-user@ip-172-31-22-105 shells]$ vi if-else2.bash
[ec2-user@ip-172-31-22-105 shells]$ ./if-else2.bash
file exists
[ec2-user@ip-172-31-22-105 shells]$ cat if-else2.bash
#!/bin/bash
if [ -e /home/ec2-user/shells/script2.bash ]
then
        echo "file exists"
else
        echo "file dont exists"
fi

[ec2-user@ip-172-31-22-105 shells]$
```

Start the Bash script.

Clear the terminal screen so that the previous output is removed.

Check whether the file /home/ec2-user/shells/scripts2.bash exists.

If the file exists, display "File exist".

If the file does not exist, execute the else part and display "File does not exist"

End the IF-ELSE statement and finish the script.

## 14.3 CASE SCRIPT

```
[ec2-user@ip-172-31-22-105 shells]$ vi case.bash
[ec2-user@ip-172-31-22-105 shells]$ cat case.bash
#!/bin/bash
echo
echo Please choose on option
echo
echo 'a=Display date and time'
echo 'b=List file and directories'
echo 'c=list users logged in'
echo 'd=check system uptime'
read choices
case $choices in
        a)date;;
        b)ls;;
        c)who;;
        d)uptime;;
        *) echo invalid choice -Bye
esac

[ec2-user@ip-172-31-22-105 shells]$ chmod 777 case.bash
[ec2-user@ip-172-31-22-105 shells]$ ./case.bash

Please choose on option

a=Display date and time
b=List file and directories
c=list users logged in
d=check system uptime
c
ec2-user pts/0        2026-10-16 08:52 (13.239.158.5)
```

```
c=list users logged in
d=check system uptime
c
ec2-user pts/0        2026-10-16 08:52 (13.239.158.5)
[ec2-user@ip-172-31-22-105 shells]$ ./case.bash

Please choose on option

a=Display date and time
b=List file and directories
c=list users logged in
d=check system uptime
d
 09:13:27 up 4 days,  1:38,  1 user,  load average: 0.00, 0.00, 0.00
[ec2-user@ip-172-31-22-105 shells]$
```

1. Start the Bash script.
2. Display a message asking the user to choose an option.
3. Display the four available options:
    - a → Display date and time
    - b → List files and directories
    - c → Display users currently logged in
    - d → Display system uptime
4. Take input from the user.
   The user's selected option is stored for checking.
5. Start the CASE statement to compare the user's choice with the available options.
6. If the user selects a, display the current date and time.
7. If the user selects b, display the files and directories in the current location.
8. If the user selects c, display the users who are currently logged into the system.
9. If the user selects d, display how long the system has been running.
10. If the user enters anything other than a, b, c, or d, display an "Invalid choice" message.
11. End the CASE statement and finish the script.

## 14.4 FOR LOOP SCRIPT

```
[ec2-user@ip-172-31-22-105 shells]$ vi for.bash
[ec2-user@ip-172-31-22-105 shells]$ ./for.bash
welcome 1 times
welcome 2 times
welcome 3 times
welcome 4 times
welcome 5 times
welcome 6 times
welcome 7 times
welcome 8 times
welcome 9 times
welcome 10 times
welcome 11 times
welcome 12 times
welcome 13 times
welcome 14 times
welcome 15 times
welcome 16 times
welcome 17 times
welcome 18 times
welcome 19 times
welcome 20 times
[ec2-user@ip-172-31-22-105 shells]$ cat for.bash

#!/bin/bash
for i in {1..20}
do
        echo "welcome $i times"
done
```

1. Start the Bash script.
2. Create a for loop with the values 1, 2, 3, 4, and 5.
3. The loop takes the first value, 1, and stores it in the loop variable.
4. Display the welcome message with the current value.
5. The loop moves to the next value, 2, and displays the message again.
6. The same process continues for 3, 4, and 5.
7. After processing the value 5, there are no more values left.
8. The loop ends and the script finishes.

## 14.5 DO-WHILE SCRIPT

```
[ec2-user@ip-172-31-22-105 shells]$ vi do-while.bash
[ec2-user@ip-172-31-22-105 shells]$ ./do-while.bash

10 seconds left to stop this process


9 seconds left to stop this process


8 seconds left to stop this process


7 seconds left to stop this process


6 seconds left to stop this process


5 seconds left to stop this process


4 seconds left to stop this process


3 seconds left to stop this process


2 seconds left to stop this process
```

```
2 seconds left to stop this process


1 seconds left to stop this process


process is stopped!!!

[ec2-user@ip-172-31-22-105 shells]$ cat do-while.bash
#!/bin/bash
count=0
num=10
while [ $count -lt 10 ]
do
        echo
        echo $num seconds left to stop this process $1
        echo
        sleep 1
        num=`expr $num - 1`
        count=`expr $count + 1`
done
echo
echo $1 process is stopped!!!
echo
[ec2-user@ip-172-31-22-105 shells]$
```

1. Start the Bash script.
2. Initialize count as 0.
   This variable keeps track of how many times the loop has executed.
3. Initialize num as 10.
   This represents the number of seconds remaining.
4. Check the while-loop condition:
   Is count less than 10?
5. If the condition is TRUE, enter the loop.
6. Display the number of seconds remaining along with the process name provided as an argument.
7. Wait for 1 second.
   This creates the countdown effect.
8. Decrease the value of num by 1.
   For example: 10 → 9 → 8 → 7 → ... → 1.
9. Increase the value of count by 1.
   For example: 0 → 1 → 2 → ... → 10.

10. Check the while condition again.
    The loop continues until count reaches 10.
11. When count becomes 10, the condition becomes FALSE, so the loop stops.
12. Display the final message saying that the specified process has been stopped.

## 14.6 EXIT SCRIPT

```
[ec2-user@ip-172-31-22-105 ~]$ cd shells
[ec2-user@ip-172-31-22-105 shells]$ ls
case.bash  do-while.bash  for.bash  if-else.bash  if-else2.bash  script1.bash  script2.bash  script3.bash  script4.bash  script5.bash
[ec2-user@ip-172-31-22-105 shells]$ vi exit.bash
[ec2-user@ip-172-31-22-105 shells]$ pwd
/home/ec2-user/shells
[ec2-user@ip-172-31-22-105 shells]$ vi exit.bash
[ec2-user@ip-172-31-22-105 shells]$ chmod 777 exit.bash
[ec2-user@ip-172-31-22-105 shells]$ ./exit.bash
-rwxrwxrwx. 1 ec2-user ec2-user 281 Oct 16 09:11 /home/ec2-user/shells/case.bash
file exist
[ec2-user@ip-172-31-22-105 shells]$
```

1. Navigated to the existing `shells` directory.
2. Listed the available shell script files in the directory.
3. Checked the current working directory to confirm the file location.
4. Created and edited the `exit.bash` script using the Vi editor.
5. Changed the file permissions to allow script execution.
6. Executed the shell script from the terminal.
7. Verified the output displayed by the script.

# Exercise: Crontab Entries

```
[ec2-user@ip-172-31-22-105 shells]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-22-105 shells]$ crontab -l
0 0 5 4 * /home/ec2-user/shells/file1.txt

0 0 5 1,6,11 * [ "$(date +\%u)" -eq 4 ] && /home/ec2-user/shells/file1.txt

5,27 9-11 * * * /home/ec2-user/shells/file1.txt

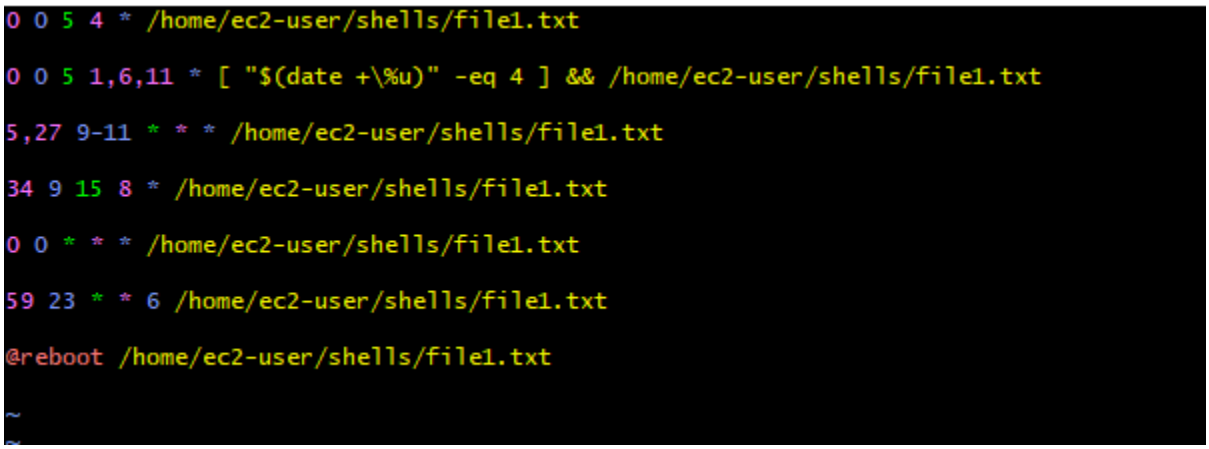
34 9 15 8 * /home/ec2-user/shells/file1.txt

0 0 * * * /home/ec2-user/shells/file1.txt

59 23 * * 6 /home/ec2-user/shells/file1.txt

@reboot /home/ec2-user/shells/file1.txt

[ec2-user@ip-172-31-22-105 shells]$
```




1. I installed the `cronie` package on my Amazon Linux 2023 EC2 instance.
2. I enabled and started the `crond` service.
3. I used my existing Bash script.
4. I opened the crontab editor using `crontab -e`.
5. I added the required cron entries according to the given scheduling requirements.
6. I saved and exited the crontab editor.
7. I verified the configured cron entries using `crontab -l`.
8. I confirmed that the `crond` service was running.
9. I configured the cron jobs to execute the script according to the specified schedules.
10. I verified the script execution by checking the output or log file, where applicable.